Optimization Methods for Data Science
Final Project
Multilayer Perceptron

Gabriele Cabibbo 2196717
Giorgio Taramanni 1961217

September 2025

1 Introduction

A **Multilayer Perceptron** (MLP) is a feedforward neural network consisting of fully connected neurons with **non-linear** activation functions and organized in **layers**.

This type of models learn a set of **weights** and **biases** to predict an output from a series of input features by minimizing a **loss function** through **back-propagation**.
Since we're aiming to predict a person's age from a picture of them, a real output, from a set of features extracted using a Convolutional Neural Network, we will learn the model's parameters by minimizing a **regularized L2** loss function:

$$E(\omega, \beta) = \frac{1}{2N} \sum_{i=1}^{n} (y_i - \hat{y_i})^2 + \lambda \sum_{l=1}^{L} ||\omega^{(l)}||^2$$

In which:

- N: Number of training samples
- y_i and $\hat{y_i}$: True target value and the value predicted for the ith training observation by the model, respectively
- λ: Regularization parameter
- $\omega^{(l)}$: Weight matrix for layer l (with L total layers)

The model's **hyperparameters**, i.e.:

- Activation functions (Sigmoid or Hyperbolic Tangent)

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Number of layers L and number of neurons in each layer N_l
- Regularization term λ

were selected using k-fold cross-validation and choosing the combination that minimized **validation loss**. We then tested the final model by analyzing the **Mean Absolute Percentage Error** (MAPE) and the **Mean Absolute Error** (MAE), which provided a more intuitive metric for evaluating the model's performance.

$$MAPE = \frac{100}{N} \sum_{i=1}^{N} \mid \frac{y_i - \hat{y_i}}{y_i} \mid \quad MAE = \frac{1}{N} \sum_{i=1}^{N} \mid \hat{y_i} - y_i \mid$$

2 Implementation

To train the model, we implemented a `NeuralNetwork` class that takes in input a list containing the number of neurons for each layer, from which we extrapolate the number of layers, the activation function and the regularization term; moreover, we initialize the weights randomly by default, but we also implemented the Glorot Normal and Uniform and the He Normal and Uniform initialization strategies. Biases are initialized as a vector of zeros. We also implemented a forward and backward method that perform forward and backward passes through the network, respectively. We used in combination with either Batch Gradient Descent, Mini-Batch Gradient Descent, or Stochastic Gradient Descent to learn the model parameters; moreover, we implemented the ADAM optimizer to adapt the learning rate to the optimization process. At last, we also implement early stopping as an additional method to prevent overfitting. We compared our custom implementation with the `minimize` function from the `scipy` library and achieved very similar results.

3 Training and Validation

We performed k-fold cross-validation with $k = 5$ folds, and used the grid of parameters shown in **Table 1** to train and evaluate our model with 384 possible hyperparameter combinations. The grid was designed to test multiple types of activation functions, different learning rate initializations, various model architectures with different numbers of hidden layers and neurons, different regularization terms (including the absence of regularization), two types of weight initializations, and two different batch sizes. Each model was trained using Mini-Batch ADAM, with its parameters set to $(\beta_1 = 0.9, \beta_2 = 0.999)$, for a maximum of 200 epochs. Early stopping with a patience of 10 was also implemented.

4 Results

Figures 1–3 show the diagnostic metrics (loss, MAPE, and MAE) comparing the behavior of hyperparameters under k-fold cross-validation and on a held-out set. Figure 1 shows that the distribution of the average cross-validation (CV) loss overlaps well with that of the fixed validation loss, indicating no symptoms of overfitting. Furthermore, the distributions are concentrated around relatively small values, suggesting that our models are generally able to achieve a low mean squared error (MSE).

Figures 2 and 3 display the same distributions for MAE and MAPE, respectively. Again, we observe that the distributions are highly concentrated near low values. Most notably, the MAE distribution is centered around a value of 7.5, which means that most of the hyperparameter combinations achieve a mean absolute error of approximately 7.5 years.

The best hyperparameter configuration is shown in **Table 2**. At the end of the first epoch the model reached a training loss of 726.02 and a validation loss

of 672.95 with very high values of MAPE (84.35%) and MAE (32.77). At epoch 101 early stopping is triggered and the model reaches a training loss equal to 48.19 and a validation loss close to 48.38 and a final MAPE of 22.92% and a MAE near 7.4.

Figures 4-6 show the evolution of the loss, MAE and MAPE throughout the training process. A smooth decrease is noticeable in both training and validation loss, as well as in the validation MAE. In contrast, the validation MAPE shows a slightly more unstable progression.

Evaluating the model on an held-out test set, we obtain the following test metrics:

- Test Loss: 45.53
- Test MAPE: 23.46%
- Test MAE: 7.21

We can, therefore, conclude that the training was effective. Although the model has a reasonable level of error (on average, it predicts an age that is 7 years off), there are no clear signs of overfitting, even when the regularization term is set to zero.

List of Figures

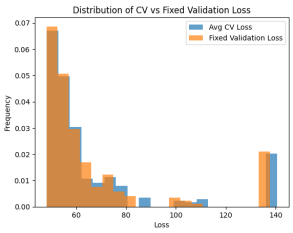

Figure 1: Distribution of Cross Validation Losses

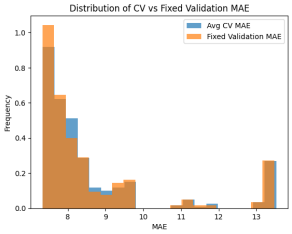

Figure 2: Distribution of Cross Validation MAEs

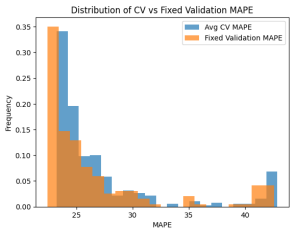

Figure 3: Distribution of Cross Validation MAPEs

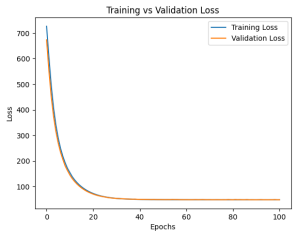

Figure 4: Train/Val Loss through Epochs

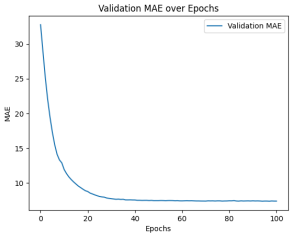

Figure 5: Validation MAE through Epochs

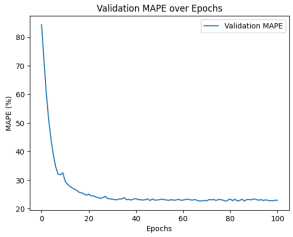

Figure 6: Validation MAPE through Epochs

List of Tables

Table 1: Hyperparameter Grid

Hyperparameter	**Values in the Grid**
Activation Function	Hyperbolic Tangent, Sigmoid
Neurons	[$N_{features}$, 8, 16, 32, 64, 1], [$N_{features}$, 8, 16, 32, 1], [$N_{features}$, 8, 16, 1], [$N_{features}$, 16, 16, 16, 16, 1]
Initialization Strategy	Glorot Normal, He Uniform
Learning Rate	0.01, 0.001, 0.005
Regularization Term	0, 0.25, 0.5, 1
Batch Size	64, 128

Table 2: Hyperparameter Configuration for the final model

Hyperparameter	**Value**
Activation Function	Sigmoid
Neurons	$[N_{features}, 8, 16, 1]$
Initialization Strategy	He Uniform
Learning Rate	0.001
Regularization Term	0
Batch Size	64

Table 3: Best Model Performance

Best Model Performance	
Final Train Loss	48.19
Final Test Loss	45.53
Final Train MAPE	22.92%
Final Test MAPE	23.46%
Final Train MAE	7.38
Final Test MAE	7.21
Optimization Time	4.02 s